

bojian.li-Hackathon文档

- [bojian.li-Hackathon文档](#)
 - [功能概述](#)
 - [采用语言模型](#)
 - [mistralai/Mixtral-8x7B-Instruct-v0.1](#)
 - [meta-llama/Meta-Llama-3.1-70B-Instruct-Turbo](#)
- [模型代码概述](#)
 - [提取会话文案-预检模版缺失项](#)
 - [提供URL地址和goal内容提取](#)
 - [提供URL地址和goal自动填充表单字段](#)
 - [FastAPI 提供服务端口](#)
- [编码辅助GitHub Copilot&文档输出](#)

功能概述

- [提交goal](#)¹ 智能体生成任务流
- [提交goal](#)¹ 提取页面内容（如表单项、页面数据）
- [提交goal](#)¹ 对页面进行操作（如筛选、点击、输入等）
- [数据校验对比提取缺失数据](#)
- [本地部署&任务唤醒本地浏览器拉取数据](#)
- [FastAPI 提供服务端口](#)

Git仓库: <https://github.com/bojian69/sentient-ai/tree/bojian-hackathon>

采用语言模型

[mistralai/Mixtral-8x7B-Instruct-v0.1](#)

专门设计用于处理自然语言指令。它可以理解并生成与用户输入相关的文本内容。该模型的主要功能包括：

自然语言理解：能够理解用户输入的自然语言指令。

文本生成：根据输入生成相关的文本内容。

对话管理：可以用于对话系统，生成对话回复。

使用功能点：

- 自然语言处理优点，提取指定模版数据；校验缺失并反馈给使用方

meta-llama/Meta-Llama-3.1-70B-Instruct-Turbo

meta 开发的大型语言模型。该模型拥有 700 亿个参数，能够理解并生成类似人类的文本。

主要特点：

高性能：拥有 700 亿个参数，能够生成连贯且上下文相关的文本。

指令跟随："Instruct-Turbo" 变体经过微调，能够遵循指令并提供详细的响应。

多功能性：适用于广泛的应用，包括文本生成、摘要、翻译等。

数据提取：从非结构化文本数据中提取结构化信息。

使用功能点：

-指令跟随：通过指令填写表单动作

-数据提取：提交URL地址及数据json模版，通过其智能体生成任务流进行数据提取

模型代码概述

提取会话文案-预检模版缺失项

```
message:"I am Liu Ting, my phone number is+61 401 234 567, and my email address is 'liuting@domain.com' The form items require last name, first name, email, and phone number to extract the required content from the form"
```

```
async def test_auto_fill_form_data(self):
    try:
        # Step 1: Define the message to send to the AI model
        message = ("I am Liu Ting, my phone number is+61 401 234 567, and my email address is "
                    "liuting@domain.com The form items require last name, first name, email, and "
                    "phone number to extract the required content from the form")

        # Step 2: Send the message to the AI model and get the response
        response = await self.async_client.chat.completions.create(
            model="mistralai/Mixtral-8x7B-Instruct-v0.1",
            messages=[{"role": "user", "content": message}],
        )

        # Step 3: Extract the form data from the response
        form_data = response.choices[0].message.content
        print("Form Data:", form_data)

    except Exception as e:
        print(e)
```

Step 3: response

Sure, I can extract the required content from the form using the provided information. Here's the extracted data:

Last name: Liu
First name: Ting
Email: liuting@domain.com
Phone number: +61 401 234 567

提供URL地址和goal内容提取

1, dome: 定义goal¹ 进行任务操作

```
message: f'Open the {web_url} and retrieve the fields name for this page form'
```

```
async def test_auto_fill_form(self):
    try:
        web_url = 'https://enpc1.uhomes.com/hackson'
        message = f'Open the {web_url} and retrieve the fields name for this
page form'

        result = await sentient.invoke(
            goal=message,
            provider="together",
            model="meta-llama/Meta-Llama-3.1-70B-Instruct-Turbo")

        print(result)

    except Exception as e:
        print(e)
```

2, AI模型通过智能体分析goal, 拆分任务流

```
=====
Current State: agentq_base
Agent: sentient
Current Thought: I see that we are currently on the uhomes.com website, specifically on the page https://enpc1.uhomes.com/hackson. The objective is to open the page https://enpc1.uhomes.com/hackson and i
Plan:
3. Retrieve the names of the fields in the form
Current Task: Retrieve the names of the fields in the form
Completed Tasks:
[✓] 1. Navigate to https://enpc1.uhomes.com/hackson
[✓] 2. Extract field names from the rental application form
[✓] 3. Retrieve the names of the fields in the form
=====
Base Agent Q has updated the memory.

=====
Objective Completed!
=====
Final Response:
The fields in the rental application form on the
https://enpc1.uhomes.com/hackson page are: First Name, Last Name, Birthday,
Email, Phone Number, University Name, and Nationality.
=====
The fields in the rental application form on the https://enpc1.uhomes.com/hackson page are: First Name, Last Name, Birthday, Email, Phone Number, University Name, and Nationality.
tearDown: end
```

3, 任务流执行过程中会衍生出页面文件数据logFields; json文件可以二次分析页面数据
[json_accessibility_dom.json](#)

[json_accessibility_dom.json](#)

提供URL地址和goal自动填充表单字段

```
async def test_auto_fill_form(self):
    try:
        web_url = 'https://enpc1.uhomes.com/hackson'
        message = (f'open the {self.web_url};'
                   f'fill the form fields with the following information: '
                   f'First Name:bojian, Last Name:li,'
                   f'Email:boj@uhomes.com, Phone Number:+86 13011070322, '
                   f'Additional Notes Beijing'
                   f'click the submit button, screenshot this page image '
                   f'base64 format.')
```

1, dome: 定义goal进行任务操作

```
message = (f'open the {self.web_url};'
           f'fill the form fields with the following information: First Name:bojian, Last '
           f'Name:li,'
           f'Email:boj@uhomes.com, Phone Number:+86 13011070322, Additional Notes Beijing'
           f'click the submit button, screenshot this page image base64 format.')
```

2, AI模型通过智能体分析goal, 拆分任务流

```
✓ Tests passed: 1 of 1 test - 3 min 12 sec
=====
Current State: agentq_base
Agent: sentient
Current Thought: Based on the current page DOM and URL, I can see that the form fields have been filled in with the provided information. Now, I need to click the submit button and then take a
Plan:
1. Go to https://enpc1.uhomes.com/hackson
2. Fill in the First Name field
3. Fill in the Last Name field
4. Fill in the Email field
5. Fill in the Phone Number field
6. Click the submit button
7. Take a screenshot of the page in image base64 format
Current Task: Click the submit button
Completed Tasks:
[✓] 1. Go to https://enpc1.uhomes.com/hackson
[✓] 2. Fill in the First Name field
[✓] 3. Fill in the Last Name field
[✓] 4. Fill in the Email field
[✓] 5. Fill in the Phone Number field
[✓] 6. Click the submit button
=====
Base Agent Q has updated the memory.

=====
Objective Completed!
=====
Final Response:
The form has been submitted successfully and the submitted information matches
the provided information.
=====
tearDown: end
```

3，任务流执行结束返回正式结果

chrome isn't your default browser

[Set as default](#)

Rental Application Form

Please fill out all required information below

Personal Information

* First Name

* Last Name

* Email

* Phone Number

Additional Information

Additional Notes

[Clear Form](#)[Submit Application](#)

The form has been submitted successfully and the submitted information matches the provided information.

FastAPI 提供服务端口

```
from fastapi import FastAPI, HTTPException, Form, Request
from pydantic import BaseModel
import uuid, os, json, datetime
from together import AsyncTogether
from sentient import sentient
from dotenv import load_dotenv
```

```

import redis.asyncio as aioredis

load_dotenv()

app = FastAPI()

redis_prefix = 'selenium:chat:message'

class RequestData(BaseModel):
    goal: str
    model: str

@app.get("/")
async def read_root():
    return {"message": "Hello, World"}

@app.get("/chat/async/models")
async def model():
    return {'models': [
        {'model': 'mistralai/Mixtral-8x7B-Instruct-v0.1', 'label': 'Mixtral-8x7B-Instruct-v0.1:自然语言处理模型'},
        {'model': 'meta-llama/Meta-Llama-3.1-70B-Instruct-Turbo', 'label': 'Llama-3.1-70B-Instruct-Turbo:指令模型'},
    ]}

@app.post("/chat/async/submit")
async def submit(request: Request, goal: str = Form(...), model: str = Form(...)):
    # 打印请求的内容
    print(await request.form())
    uuid_str = str(uuid.uuid4())
    print('TOGETHER_API_KEY:', os.environ.get("TOGETHER_API_KEY"))
    async_client = AsyncTogether(api_key=os.environ.get("TOGETHER_API_KEY"))
    print({
        'goal': goal,
        'model': model,
    })
    message = ''
    # 切换检索模型
    if model == 'mistralai/Mixtral-8x7B-Instruct-v0.1':
        try:
            response = await async_client.chat.completions.create(
                model="mistralai/Mixtral-8x7B-Instruct-v0.1",
                messages=[{"role": "user", "content": goal}],
            )
            message = response.choices[0].message.content
        except Exception as e:
            message = str(e)

```

```

        print(message)
    elif model == 'meta-llama/Meta-Llama-3.1-70B-Instruct-Turbo':
        try:
            message = await sentient.invoke(
                goal=goal,
                provider="together",
                model="meta-llama/Meta-Llama-3.1-70B-Instruct-Turbo")
        except Exception as e:
            message = str(e)
            print(message)

```

格式化返回数据

```

response = {
    'uuid': uuid_str,
    'goal': goal,
    'message': message,
    'model': model,
    'timestamp': datetime.datetime.now().timestamp()
}

```

message 数据存入 Redis

```

redis = aioredis.from_url('redis://localhost', decode_responses=True)
await redis.set(
    name=f'{redis_prefix}:{uuid_str}',
    value=json.dumps(response, ensure_ascii=False),
    ex=3600 * 24 * 7
)
await redis.aclose()

return response

```

@app.post("/chat/async/message")

async def message(uuid: str = Form(...)):

message 数据存入 Redis

```

redis = aioredis.from_url('redis://localhost', decode_responses=True)
response = await redis.get(name=f'{redis_prefix}:{uuid}')
await redis.aclose()

```

if response is None:

```

    raise HTTPException(status_code=404, detail="Message not found")

```

```

response = json.loads(response)

```

判断timestamp是否存在

if 'timestamp' not in response:

```

    response['timestamp'] = datetime.datetime.now().timestamp()

```

```

return {

```

```

    'goal': response['goal'],
    'message': response['message'],
    'model': response['model'],
    'uuid': uuid,

```



```
        'timestamp': response['timestamp']
    }

if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=8000)
```

编码辅助GitHub Copilot&文档输出

 araleeat



python 测试AI自动填写表单的方法，如何设计方法体

 GitHub Copilot



以下是一个详细的设计步骤和代码示例，用于测试AI自动填写表单的方法。

设计步骤

1. 设置环境：加载环境变量，初始化异步客户端。
2. 定义测试类：继承`IsolatedAsyncioTestCase`，并在`asyncSetUp`中初始化异步客户端。
3. 编写测试方法：
 - 发送请求以获取表单数据。
 - 使用AI模型生成填写表单的内容。
 - 验证生成的内容是否符合预期。
4. 清理资源：在`asyncTearDown`中清理资源。

Plan

1. Parse the `form_data` string to extract the required fields: Last name, First name, Email, and Phone number.
2. Use regular expressions to find and extract these fields from the `form_data` string.

Code

```
import re

# Example form_data string
form_data = response.choices[0].message.content

# Define regular expressions to extract the required fields
last_name_pattern = re.compile(r"Last name:\s*(\w+)")
first_name_pattern = re.compile(r"First name:\s*(\w+)")
email_pattern = re.compile(r"Email:\s*([\w\.-]+@[\w\.-]+)")
phone_number_pattern = re.compile(r"Phone number:\s*(\+?\d[\d\s-]+)")

# Extract the fields using the defined patterns
```

`asyncio.run()` cannot be called from a running event loop

To resolve the issue of `asyncio.run()` being called from a running event loop, you can use `await` directly instead of `asyncio.run()`. Here is the updated `test_auto_fill_form` method:

```
async def test_auto_fill_form(self):
    try:
        web_url = 'https://enpc1.uhomes.com/hackson'
        message = f'Open the {web_url} and retrieve the fields name for this page for

        result = await sentient.invoke(
            goal=message,
            provider="together",
            model="meta-llama/Meta-Llama-3.1-70B-Instruct-Turbo"
        )

        print(result)

    except Exception as e:
        print(e)
```



🔗 [What other methods can I use to handle asynchronous tasks in Python?](#)

`sentient.invoke` 方法用于调用 AI 模型来完成特定的目标。以下是其用法的解释：

1. 定义目标：首先需要定义一个目标（goal），这是你希望 AI 模型完成的任务。
2. 指定提供者和模型：需要指定使用哪个提供者（provider）和模型（model）。
3. 调用方法：使用 `await` 关键字调用 `sentient.invoke` 方法，并传入目标、提供者和模型。

示例代码：

```
result = await sentient.invoke(
    goal="你的目标描述",
    provider="提供者名称",
    model="模型名称"
)
```

在你的代码中，`sentient.invoke` 方法被用来打开一个网页并检索该页面表单的字段名称。

```
response = await sentient.invoke( goal=message, provider="together", model="meta-llama/Meta-Llama-3.1-70B-Instruct-Turbo")
```

```
await self.result_to_file(response, 'test_auto_fill_form_response')
```

一次执行是否会有多种结果

 GitHub Copilot



`sentient.invoke` 方法一次执行通常会返回一个结果。如果你希望获取多个结果，可以在调用该方法时指定相应的参数，或者在循环中多次调用该方法。

以下是一个示例，展示了如何在循环中多次调用 `sentient.invoke` 方法以获取多个结果：

```
results = []
for _ in range(5): # 假设我们希望获取5个结果
    response = await sentient.invoke(
        goal=message,
        provider="together",
        model="meta-llama/Meta-Llama-3.1-70B-Instruct-Turbo"
    )
    results.append(response)
    await self.result_to_file(response, f'test_auto_fill_form_response_{_}')
```

这样，每次调用 `sentient.invoke` 方法都会返回一个结果，并将其保存到文件中。

 GitHub Copilot



要使用 Selenium 创建一个简单的对外 API 窗口，可以使用 Flask 框架来创建一个 Web 服务，并在其中集成 Selenium 进行浏览器自动化操作。以下是详细的步骤和代码示例：

1. 安装 Flask 和 Selenium:

```
pip install flask selenium
```

2. 创建一个 Flask 应用，并在其中使用 Selenium 进行浏览器操作。

3. 定义一个 API 端点，处理外部请求并返回结果。

以下是一个示例代码：

步骤1: 创建 Flask 应用

以下是一个示例代码：

步骤1: 创建 Flask 应用

```
from flask import Flask, jsonify, request
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager

app = Flask(__name__)

@app.route('/api/perform_action', methods=['POST'])
def perform_action():
    data = request.json
    url = data.get('url')

    # 使用 Selenium 打开浏览器并访问指定 URL
    options = webdriver.ChromeOptions()
    options.add_argument('--headless') # 无头模式
    driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=options)

    driver.get(url)
    title = driver.title
    driver.quit()

    return jsonify({'title': title})

if __name__ == '__main__':
    app.run(debug=True)
```


步骤2: 运行 Flask 应用

```
python app.py
```

步骤3: 发送请求到 API

可以使用 `curl` 或 Postman 发送 POST 请求到 `http://127.0.0.1:5000/api/perform_action`, 请求体包含要访问的 URL, 例如:

```
{  
  "url": "https://www.example.com"  
}
```

API 将返回页面的标题作为响应。

> Using 1 Reference

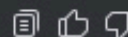


araleeat

能否用 uvicorn 创建个对外的API端口



GitHub Copilot



计划:

1. 安装 `fastapi` 和 `uvicorn`。
2. 创建一个简单的 FastAPI 应用。
3. 定义一个 API 端点。
4. 使用 `uvicorn` 运行 FastAPI 应用。

```
pip install fastapi uvicorn
```

创建 FastAPI 应用

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/api/hello")
```

```
async def read_root():
```

```
    return {"message": "Hello, World"}
```

使用 uvicorn 运行应用

```
if __name__ == "__main__":
```

```
    import uvicorn
```

```
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

运行应用：

```
python app.py
```

现在可以通过访问 <http://127.0.0.1:8000/api/hello> 来访问 API。

> Using 1 Reference

Flask 和 FastAPI 是两个流行的 Python Web 框架，它们各有优缺点。以下是它们的主要区别：

1. 性能：

- **Flask**: Flask 是一个轻量级的 WSGI 框架，性能相对较低，适合小型应用和快速开发。
- **FastAPI**: FastAPI 是一个基于 ASGI 的框架，性能非常高，适合构建高性能的异步 Web 应用。

2. 异步支持：

- **Flask**: Flask 本身不支持异步编程，但可以通过第三方库（如 `gevent` 或 `eventlet`）实现。
- **FastAPI**: FastAPI 原生支持异步编程，使用 `async` 和 `await` 关键字。

3. 类型注解：

- **Flask**: Flask 不强制使用类型注解，开发者可以自由选择是否使用。
- **FastAPI**: FastAPI 强烈依赖 Python 的类型注解，能够自动生成 API 文档（如 OpenAPI 和 JSON Schema）。

4. API 文档：

- **Flask**: Flask 需要手动编写 API 文档，通常使用 `flasgger` 或 `apispec` 等库。
- **FastAPI**: FastAPI 自动生成 API 文档，支持 Swagger UI 和 ReDoc。

5. 依赖注入：

- **Flask**: Flask 没有内置的依赖注入机制，但可以通过 Flask-Injector 等库实现。
- **FastAPI**: FastAPI 内置了依赖注入系统，简化了依赖管理。

6. 启动方式：

- **Flask**: Flask 通常使用内置的开发服务器或通过 `gunicorn` 等 WSGI 服务器部署。
- **FastAPI**: FastAPI 通常使用 `uvicorn` 或 `hypercorn` 等 ASGI 服务器部署。

总结:

- **Flask**: 适合小型项目、快速开发和简单的同步 Web 应用。
- **FastAPI**: 适合高性能、异步 Web 应用和需要自动生成 API 文档的项目。

> Using 1 Reference



araleeat

python项目查看包信息



GitHub Copilot



要查看 Python 项目中已安装的包信息，可以使用以下几种方法：

1. 使用 **pip list** 命令：在终端中运行以下命令，可以列出当前环境中所有已安装的包及其版本信息：

```
pip list
```

2. 使用 **pip freeze** 命令：该命令会列出所有已安装的包及其版本信息，输出格式适合写入 **requirements.txt** 文件：

```
pip freeze
```

3. 查看 **requirements.txt** 文件：如果项目中有 **requirements.txt** 文件，可以直接查看该文件，了解项目所需的包及其版本信息。
4. 使用 **pkg_resources** 模块：可以在 Python 脚本中使用 **pkg_resources** 模块来列出已安装的包信息：

```
import pkg_resources

installed_packages = pkg_resources.working_set
for package in installed_packages:
    print(f"{package.key}=={package.version}")
```

fastapi post 传参并请求 goal 和 model type

计划:

1. 创建一个 FastAPI 应用。
2. 定义一个 POST 端点，接收 **goal** 和 **model_type** 参数。
3. 返回接收到的参数作为响应。

代码:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class RequestData(BaseModel):
    goal: str
    model_type: str

@app.post("/api/submit")
async def submit_data(data: RequestData):
    return {"goal": data.goal, "model_type": data.model_type}

if __name__ == "__main__":
```

运行应用：

```
python app.py
```

现在可以通过发送 POST 请求到 `http://127.0.0.1:8000/api/submit``，并在请求体中包含 `goal` 和 `model_type`` 参数来访问 API。

> Using 4 References

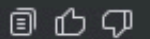


araleeat

FastAPI() 有没有同步返回的方法？



GitHub Copilot



是的，FastAPI 支持同步返回的方法。你可以在定义路由处理函数时不使用 `async` 关键字，这样函数将以同步方式执行。

以下是一个示例：

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class RequestData(BaseModel):
```

```
    goal: str
```

```
    model: str
```

```
@app.post("/api/submit")
```

```
if __name__ == "__main__":  
    import uvicorn  
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

在这个示例中，`read_root` 和 `submit_data` 函数都是同步的。

> Using 1 Reference

 araleeat

生成UUID方法

 GitHub Copilot

计划：

1. 导入 `uuid` 模块。
2. 定义一个函数来生成 UUID。

代码：

```
import uuid  
  
def generate_uuid():  
    return str(uuid.uuid4())
```

这个函数将生成一个新的 UUID 并将其转换为字符串。你可以在需要生成 UUID 的地方调用这个函数。

To fix the **422 Unprocessable Entity** error, you need to modify the **submit_data** function to accept the **uuid** parameter as a query parameter instead of a request body. Here is the updated code:

```
@app.get("/chat/async/message")
async def get_message(uuid: str):
    return {"uuid": uuid}
```

This change ensures that the **uuid** parameter is correctly parsed from the query string.

> Using 4 References

 araleeat

INFO: 127.0.0.1:55861 - "GET /chat/async/message?uuid=bojian-uuid HTTP/1.1" 422 Unprocessable Entity

用中文回答原因

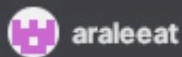
 GitHub Copilot

出现 **422 Unprocessable Entity** 错误的原因是因为在 GET 请求中，FastAPI 期望从查询参数中获取 **uuid**，而不是从请求体中获取。

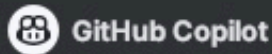
你需要修改 **message** 函数，使其从查询参数中获取 **uuid**。以下是修改后的代码：

```
@app.get("/chat/async/message")
async def message(uuid: str):
    return {"uuid": uuid}
```



araleeat

INFO: 127.0.0.1:56207 - "GET /chat/async/message?uuid=bojian-uuid HTTP/1.1" 422 Unprocessable Entity



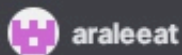
出现 **422 Unprocessable Entity** 错误的原因是因为在 GET 请求中，FastAPI 期望从查询参数中获取 **uuid**，而不是从请求体中获取。

你需要确保在 GET 请求中正确传递 **uuid** 参数。以下是修改后的代码：

```
@app.get("/chat/async/message")
async def message(uuid: str):
    return {"uuid": uuid}
```

这样，**uuid** 参数将正确地从查询字符串中解析。

> Using 1 Reference



araleeat

INFO: 127.0.0.1:58108 - "POST /chat/async/message HTTP/1.1" 422 Unprocessable Entity

```
"detail": [
  {
    "type": "model_attributes_type",
    "loc": [
      "body"
    ],
    "msg": "Input should be a valid dictionary or object to extract fields from",
    "ctx": {
      "error": {}
    }
  }
]
```


出现 **422 Unprocessable Entity** 错误的原因是因为 FastAPI 期望从请求体中获取 JSON 格式的数据，而你发送的是 **multipart/form-data** 格式的数据。

你需要修改 **message** 函数，使其能够处理 **multipart/form-data** 格式的数据。以下是修改后的代码：

```
from fastapi import FastAPI, HTTPException, Form
from pydantic import BaseModel
import uuid

app = FastAPI()

class RequestData(BaseModel):
    goal: str
    model: str

@app.get("/")
async def read_root():
    return {"message": "Hello, World"}

@app.post("/chat/async/submit")
async def submit(data: RequestData):
    uuid_str = str(uuid.uuid4())
    return {"goal": data.goal, "model": data.model, "uuid": uuid_str}

@app.post("/chat/async/message")
async def message(goal: str = Form(...), model: str = Form(...)):
    return {"goal": goal, "model": model}

if __name__ == "__main__":
```


这样，`message` 函数将能够处理 `multipart/form-data` 格式的数据。

> Using 1 Reference

 araleeat

```
127.0.0.1:58696 - "POST /chat/async/submit HTTP/1.1" 422 Unprocessable Entity
{"detail":[{"type":"model_attributes_type","loc":["body"],"msg":"Input should be a valid dictionary or object t
from","input":"-----843033091841127829886326\r\nContent-Disposition: form-data;
name="goal"\r\n\r\ntest goal\r\n-----843033091841127829886326\r\nContent-Disp
name="model"\r\n\r\nmistralai/Mixtral-8x7B-Instruct-v0.1\r\n-----8430330918411278
```

 GitHub Copilot

出现 `422 Unprocessable Entity` 错误的原因是因为 `FastAPI` 期望从请求体中获取 `JSON` 格式的数据，而你发送的是 `multipart/form-data` 格式的数据。

你需要修改 `submit` 函数，使其能够处理 `multipart/form-data` 格式的数据。以下是修改后的代码：

```
from fastapi import FastAPI, HTTPException, Form
from pydantic import BaseModel
import uuid

app = FastAPI()

class RequestData(BaseModel):
    goal: str
    model: str

@app.get("/")
async def read_root():
```

```
@app.get("/")
async def read_root():
    return {"message": "Hello, World"}

@app.post("/chat/async/submit")
async def submit(goal: str = Form(...), model: str = Form(...)):
    uuid_str = str(uuid.uuid4())
    return {"goal": goal, "model": model, "uuid": uuid_str}

@app.post("/chat/async/message")
async def message(goal: str = Form(...), model: str = Form(...)):
    return {"goal": goal, "model": model}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=8000)
```

这样，`submit` 函数将能够处理 `multipart/form-data` 格式的数据。

-
1. `goal`: 指你希望AI模型完成的任务 ↩ ↩2 ↩3 ↩4